LINFO/LSINC1361: Artificial Intelligence
Assignment 2: Adversarial Search (Oxono)

Achille Morenville, Alice Burlats, Amaury Fierens, Thomas Bouvencourt, Eric Piette
March 2026

Guidelines

- The report and your agent must be submitted on **Tuesday, 14th April 2026, 18h00**. Your contest agent implementation can to be submitted up to the **Tuesday, 5th May 2026, 18h00**.
- *No delay* will be tolerated.
- *Document* your source code (at least the difficult or more technical parts of your programs). Python docstrings for important classes, methods and functions are also welcome.
- Copying code or answers from other groups (or from the internet) is strictly forbidden. Each source of inspiration must be clearly indicated.
- Source code shall be submitted on the online *INGInious* system. Only programs submitted via this procedure will be graded. No program sent by email will be accepted.
- Respect carefully the *specifications* given for your program (arguments, input/output format, etc.) as the program testing system is *fully automated*.

During the lectures, you have explored adversarial search algorithms, including MiniMax, Alpha-Beta, and Monte-Carlo Tree Search (MCTS). In this project, you will implement and improve these algorithms to create an AI agent capable of playing the board game Oxono.

Oxono is a strategic two-player board game that offers an interesting challenge for adversarial search techniques. A detailed description of the game rules is provided below, and you may watch this short video for a quick overview of the game.

Figure 1: Oxono Game Cover.

1 Game Rules

Oxono is a strategic board game for two players, **pink** and **black**, played on a 6×6 grid. Each player has 16 pieces of their color, divided into 8 Xs and 8 Os. Two shared Totems, one X and one O, are also in play. These Totems serve as anchors that determine where pieces may be placed during each turn.

Goal of the Game

Be the first player to align 4 pieces of the same color or the same symbol, in a horizontal or vertical line.

Set Up

Both Totems (X and O) start on the two central squares of the board, as shown in Fig. 2.

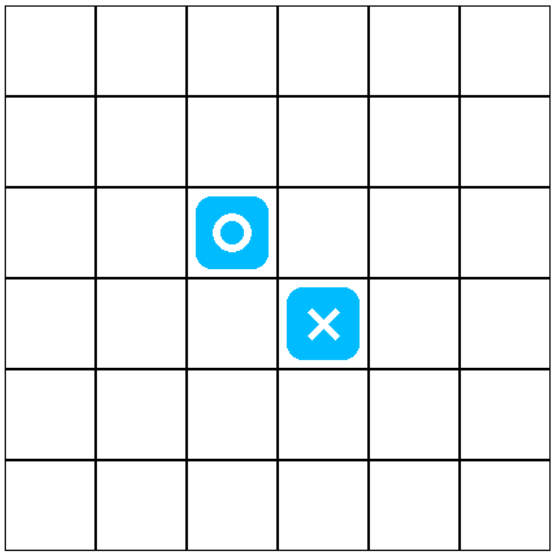

Figure 2: Initial game setup. Totems are represented in light blue.

Turn Structure

The pink player goes first. On each turn, a player must perform the following two steps:

1. **Move a Totem**

   Move one Totem of their choice (X or O) horizontally or vertically by at least one square and any number of squares. A Totem cannot pass over any occupied square (piece or the other Totem), it may only travel over empty squares. Fig. 3 illustrates the possible movements of the O Totem.

2. **Place a Piece**

   Place one of their own pieces matching the symbol of the Totem just moved, on any free square horizontally or vertically adjacent to the Totem's new position (up to 4 possible squares). Fig. 4 shows the possible placements after moving the X Totem.

A player may move a Totem only if they still have pieces of the corresponding symbol available in their reserve.

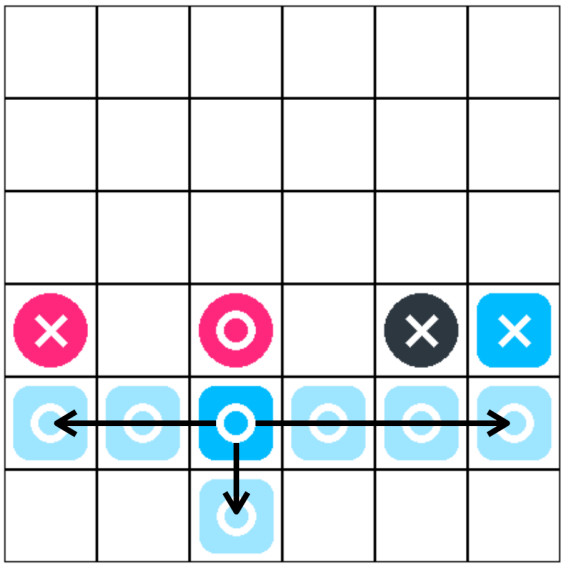

Figure 3: Possible moves for the Totem O.

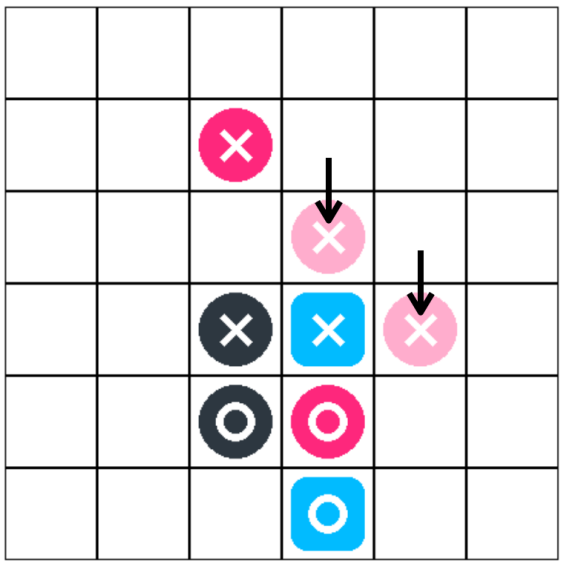

Figure 4: Possible placement of pieces after the movement of Totem X.

Special Cases

Surrounded Totem A Totem is surrounded when every adjacent square (horizontally and vertically) is occupied. In this case, the Totem may jump over one or more consecutive occupied squares in a straight line and must land on the first free square beyond them. The player then places a matching piece following the usual placement rules. Fig. 5 illustrates this rule.

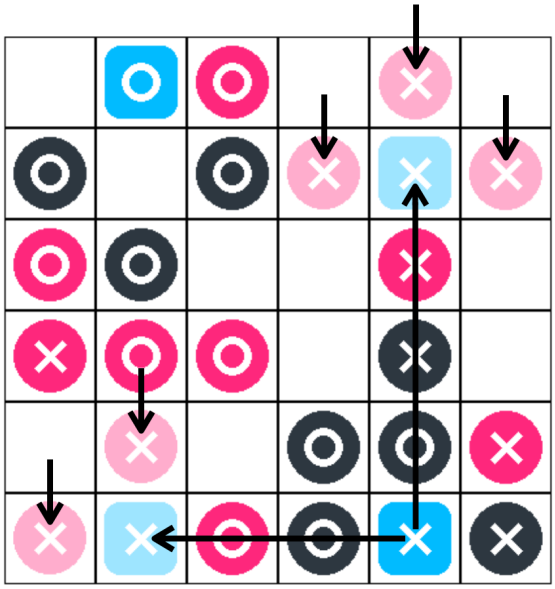

Figure 5: Example of movement for a surrounded Totem and the subsequent piece placement.

Surrounded destination If a Totem moves to a surrounded square, the player may place their matching piece on any free square on the board.

Fully blocked row and column If every square in both the row and column of a surrounded Totem is occupied, the Totem my be moved to any free square on the board. The player then places a matching piece following the normal placement rules, or if the destination is surrounded, on any free square on the board.

End of Game

The game ends immediately when a player aligns 4 pieces of the same color (regardless of symbol) or 4 pieces of the same symbol (regardless of color), horizontally or vertically. The player who places the 4th piece forming the alignment wins, even if the opponent has more pieces of their color in that line. Totems do not count toward any alignment.

If all 32 pieces are placed without any winning alignment, the game ends in a draw.

2 Provided Code

The provided codebase includes the implementation of the Oxono game, files for developing your AI agents, and testing them against each other.

Game Logic

The core game logic is implemented in the `oxono.py` file.

The `State` class represents a complete state of Oxono. The `Game` class implements the game rules and provides five key methods:

- `Game.to_move(state)`: Returns the player whose turn it is to move.

- `Game.actions(state)`: Returns the list of all legal actions available in the given state.
- `Game.apply(state, action)`: Applies an action to the state. The state object is modified directly.
- `Game.is_terminal(state)`: Returns whether the state satisfies the ending conditions.
- `Game.utility(state, player)`: Computes the utility of the state for a specified player.

The game state representation is described in detail in the documentation of the `oxono.py` file.

Agents

The `agent.py` file defines the interface used to implement AI agents.

To create an agent, extend the `Agent` class and implement the `act(state, remaining_time)` method. This method receives the current game state and the remaining time (in seconds) as inputs, and it should return the action to be taken by the agent.

A basic random agent is provided in `random_agent.py`, which you can use as a starting point. This random agent selects moves uniformly at random from the list of legal actions, serving as a trivial baseline for testing and comparison.

Game Managers

The codebase includes two additional files to simplify testing and evaluation:

- `manager.py`: Allows you to run games between agents and save the results to log files.
- `visual_manager.py`: This file provides a graphical interface for visualizing and interacting with the game. You can use it to play against your agent, watch two agents compete, or simply observe the game state in a more interactive manner.
- `replayer.py`: Provides a graphical interface for replaying previously recorded games.

The documentation within these files provides all the necessary information for their use.

Installations and Requirements

To run the codebase, ensure you have Python 3.12 installed. Install the required dependencies by running the following command in the project directory:

```
pip install -r requirements.txt
```

3 Project

The goal of this project is to develop an AI agent capable of playing Oxono.

You are free to use any adversarial search algorithm or combination of algorithms to create your agent. You may improve upon Alpha-Beta or Monte-Carlo Tree Search (MCTS) by enhancing their evaluation functions, search strategies, or other components. Alternatively, you can explore entirely different algorithms or techniques not covered in the course. The

goal is to create an agent that outperforms the baseline implementations through creativity, experimentation, and optimization.

If you find it challenging to decide where to start, here are some sources of inspiration:

- For the Alpha-Beta algorithm, you could try to improve the evaluation function, the search strategy, or the pruning techniques. There is a long list of possible improvements that are cited in the reference book[1] in the Chapters 6.2.3 to 6.3.4 and in the slides of the course. As an example, you could try to implement a transposition table, a quiescence search, or a killer move heuristic (those are just examples, do not limit yourself to those). You could also try to design a good evaluation function for the game of Oxono.
- For the MCTS algorithm, there is a long list of improvements that are described in the scientific literature. Especially in this survey paper[2]. As a starting point, you could try to improve the simulation policy, or the tree policy.
- You could also try to use neural networks to improve the evaluation function of your agent. This is a very popular approach in the field of AI and could be very interesting to try.
- ...
- And many more possibilities! This list is not exhaustive, and you are free to try anything you want, **as long as it follows the rules that will be described later**.

This project will be evaluated based on three components:

- Implementation of the agent
- Written report
- Final contest ranking

The following sections will describe these components in detail.

3.1 Implementation (40%)

Your implementation must satisfy the following constraints:

- Your agent should be implemented in **Python 3.12 or later**. Other programming languages are not allowed (for example through compiled extensions or embedded binaries). However, it is permitted to use libraries that internally rely on non-Python implementations[3].
- Your agent should extend the Agent class provided in the codebase and should implement the `act(state, remaining_time)` method.
- Each agent has a total time budget of **5 minutes per game**. If the agent exhausts its time budget before the game ends, it automatically loses the game.

[1] Russell, S. J., & Norvig, P. (2016). Artificial intelligence: a modern approach. pearson.

[2] Browne, C. B., Powley, E., Whitehouse, D., Lucas, S. M., Cowling, P. I., Rohlfshagen, P., ... & Colton, S. (2012). A survey of monte carlo tree search methods. IEEE Transactions on Computational Intelligence and AI in games, 4(1), 1-43.

[3] Libraries available on *Inginious* are *keras 3.13.1*, *numpy 2.4.1*, *pandas 2.3.3*, *scikit-learn 1.8.0*, *scipy 1.17.0*, *tensorflow 2.20.0*, and *torch 2.9.1*

- The action returned by your agent must always belong to the list returned by `Game.actions(state)`. Returning an invalid action will result in an immediate loss.
- For fairness between teams and due to hardware limitations, the use of GPUs and multithreading is not allowed. Your agent must not require more than **2 GB of RAM**.
- Your agent should be implemented in a file named: `my_agent.py`. In addition, it must contains a single implementation of an `Agent` class.
- If additional files are required, they must be placed in a folder named `files_{NOMA1}_{NOMA2}`. Ensure that all imports and file paths in your `my_agent.py` file work correctly.
- The total size of the submission after compression must not exceed **100 MB**.

Submit

Submit your agent on Inginious via this task before the **Tuesday, 14th April 2026, 18h00**. Automated tests will run your agents against one of the *Inginious* agent for 10 games.

Evaluation criteria:

- Code quality (structure, readability, documentation, and comments).
- Performance against several reference agents (random agent, greedy agent, and other agents of varying difficulty).

Notes:

- This task will also verify that your agent satisfies the constraints described above. You are encouraged to submit early to ensure that everything works correctly before the deadline. If both members of a team submit an agent, the **most recent submission** will be used for evaluation.
- The *Inginious* agent is a first challenge to beat and can give you an idea of the performance of your agent. However, these results alone are not enough to justify it.

3.2 Report (40%)

In addition to the implementation, you must submit a report describing your agent and the design decisions behind it. The report may be written in either English or French and must not exceed **five pages**. You are free to organize the report as you wish, but it must contain the information described in this section.

As a first step, provide a clear and detailed description of the design of your agent. Because this project is intentionally open-ended, your explanation should contain all the information necessary for the reader to understand how your agent works. In particular, you should describe what you implemented and justify the design choices you made. If you implemented specific heuristics, optimizations, or tricks, explain them and discuss why you believe they are effective for the game of Oxono.

Your report should address the following points:

- The specific improvements you made to the baseline algorithms.

- Any new algorithms or techniques you experimented with for Oxono.
- Approaches that you attempted but that did not work as expected.
- A clear explanation of why your final design is well suited to the game of Oxono.

To support your design choices, you must conduct a series of experiments evaluating the performance of your agent. Your experimental analysis should provide clear and quantitative evidence of the improvements made during the development process.

Design and conduct experiments to validate the performance of your agent. Your evaluation should include:

- A brief description of the experimental setup, including the number of games played, the opponents used, and any other relevant parameters. Your experiments should be sufficiently described so that they could be reproduced by another reader.
- One or more tables or graphs showing the evolution of the performance of different versions of your agent. These results should be compared against a baseline agent of your choice. Relevant metrics may include win rate, average game length, or other indicators you consider informative. The choice of the baseline should be clearly justified.
- A comparison between your final agent and several reference agents, including at least a random agent, a baseline Alpha-Beta agent, and a baseline MCTS agent. You may include additional agents if relevant.
- A discussion of the experimental results. In this discussion, interpret the observed performance, explain how the results support your design decisions, and highlight any interesting insights or trends you discovered.

For example, the following list illustrates what a thorough evaluation typically looks like. You are not required to follow it exactly, but it may help you assess the completeness of your analysis.

- Results are reported for both colors (pink and black) separately, or a sufficient number of games is played to average out a potential first-mover advantage.
- Each matchup is repeated enough times to yield statistically meaningful win rates (for example, at least 20-50 games per matchup).
- An ablation study is conducted: individual components are added or removed one at a time to isolate their contribution to performance.
- Performance is compared across multiple opponents of varying strength (such as random, greedy agent, baseline Alpha-Beta, baseline MCTS, and other meaningful agents).
- Results include not only win rates but also other informative metrics (such as average game length, time per move, draw rate).
- Experiments that did not work are reported and discussed, not just the successful ones.
- The experimental setup is described in enough detail to be reproducible by another reader.
- Results are presented in a clear visual format (table or graph), not only described in sentences.

- The discussion interprets the results rather than merely restating them (for example explaining *why* a technique helped or failed).

Submit

Submit your report on *Moodle* before **Tuesday, 14th April 2026, 18h00**.

Evaluation criteria:

- Overall quality and clarity of the report.
- Completeness of the required information.
- Quality of the explanations and justification of design choices.
- Quality and rigor of the empirical evaluation.

3.3 Contest (20%)

Now, it is time for blood and glory.

At the end of the project, a competition will be organized between all submitted agents in order to produce a leaderboard. An extended deadline is provided for the contest agent if you wish to continue improving your agent after the main project deadline.

Submit

Once you have implemented your agent with all the desired features for the contest, submit it on Inginious via this task before ***Tuesday, 5th May 2026, 18:00***.

Evaluation criterion:

- The final ranking will be determined using a rating system. Initially, all agents start with the same rating. During the competition, agents are paired using a matchmaking system that preferentially matches agents with similar ratings. The ratings are then updated according to the matches results. Once the ratings have stabilized, the leaderboard will be divided into performance groups, and agents in higher-performing groups will receive more points.

Important:

- Your contest agent must follow the same implementation rules as those specified for the first deadline.
- **This second deadline is optional.** If you do not submit a new version, the last agent submitted for the previous task will automatically be used in the competition.
- If you submit a contest agent and it performs better than the version previously submitted (based on match results), you will receive a bonus of **1 additional point**.

4 Deliverables

The project deliverables are summarized below:

Submit

- A PDF report describing your agent and experimental evaluation (submitted on *Moodle*).
- Your agent implementation (submitted on *Inginious*).
- Optionally, an improved contest agent (submitted on *Inginious*).